

Turning nine spreadsheets into a revenue forecast CloudFlow can act on

A complete walkthrough of how raw billing exports become a validated database, a leak-free machine learning feature set, a competition between seven forecasting models, and the live dashboard that CloudFlow's team actually opens every morning.

59

months of revenue history

2.18%

forecast error (MAPE)

7

models backtested

6 mo

forecast horizon

WHY THIS EXISTS

CloudFlow needed one number everyone could trust

CloudFlow sells three product lines — Core Platform, Analytics Add-on, and Automation Suite — across four pricing tiers to customers ranging from self-serve startups to negotiated enterprise contracts, in five regions. Revenue comes from new sales, renewals, expansion, and is offset by churn — and before this system existed, answering "what will next quarter look like" meant someone manually reconciling exports from billing, usage, and CRM systems.

The brief was simple to state and hard to do honestly: build a single pipeline from source data to forecast, pick a model on evidence rather than intuition, and put it in front of the team as a dashboard they'd actually use — not a one-off spreadsheet that goes stale the day someone closes it.

Six phases, one MySQL database, no shortcuts

Every stage reads from and writes to the database — the source CSVs are touched exactly once, at ingestion. Nothing downstream ever reads a spreadsheet again.

1

Ingest `src/ingest_csv.py`

Create the database, stage each of the 9 source CSVs into its own raw table, and validate row counts, referential integrity, and date ranges before anything downstream runs.

2

Process `src/process_data.py`

Clean and type the raw tables into proper dimension/fact tables and a monthly revenue summary, reconciling every dollar back to the source transactions.

3

Engineer features `src/feature_engineering.py`

Build the ML feature table — every single predictor is lagged one month behind the target, so the model can never accidentally see the future during training.

4

Evaluate `src/evaluate_model.py`

Backtest all seven candidate models with an expanding walk-forward window and score them on the same 12 held-out months.

5

Forecast `src/forecast.py`

Refit the winning model on the full history and roll it forward six months with an 80% confidence interval.

6

Serve `app.py`

A Streamlit dashboard reads only the finished output tables — never the raw data — through a single query layer.

CHOOSING A MODEL HONESTLY

The simplest model won — and that's the honest result

Rather than reaching for gradient boosting by default, seven models were backtested against the same 12 months, one month at a time, each one trained only on data that would have actually been available at that point in history.

MODEL	TYPE	MAPE	
drift	Baseline — last value + average trend	2.18%	SELECTED
holt_linear	Classical exponential smoothing	2.19%	
naive	Baseline — last observed value	2.70%	
moving_avg_3	Baseline — trailing 3-month mean	3.65%	
xgboost	Gradient-boosted trees	4.16%	
lightgbm	Gradient-boosted trees	7.84%	
seasonal_naive	Baseline — value 12 months prior	20.50%	

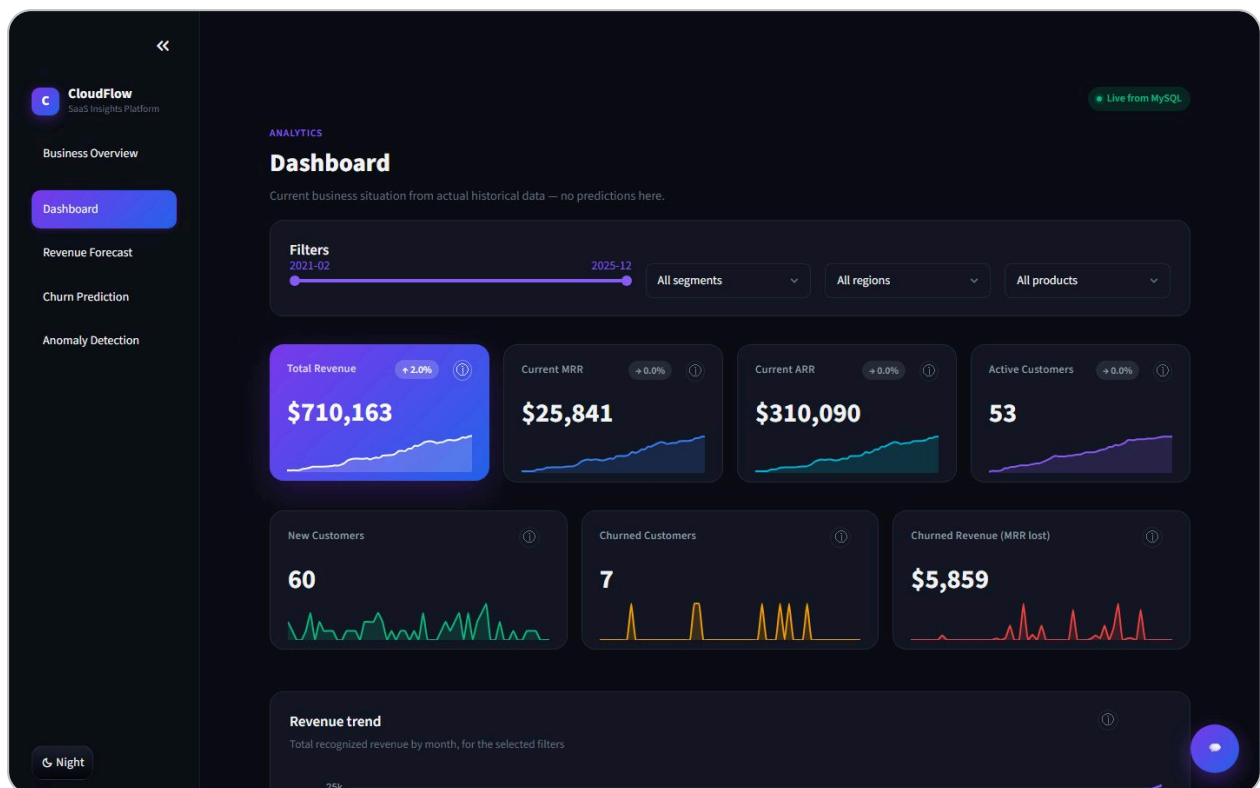
Why a simple baseline beat two gradient-boosted models: CloudFlow's 59 months of revenue are smooth and trend-dominated, without the sharp seasonality or nonlinear interactions that give tree models something to exploit — `drift` and `holt_linear` finish within a hundredth of a point of each other, while XGBoost and LightGBM trail by roughly 2x and 3.5x respectively. Reporting the honest winner — instead of quietly

defaulting to whichever model sounds most sophisticated — is the difference between a forecast you can trust and one that just looks impressive.

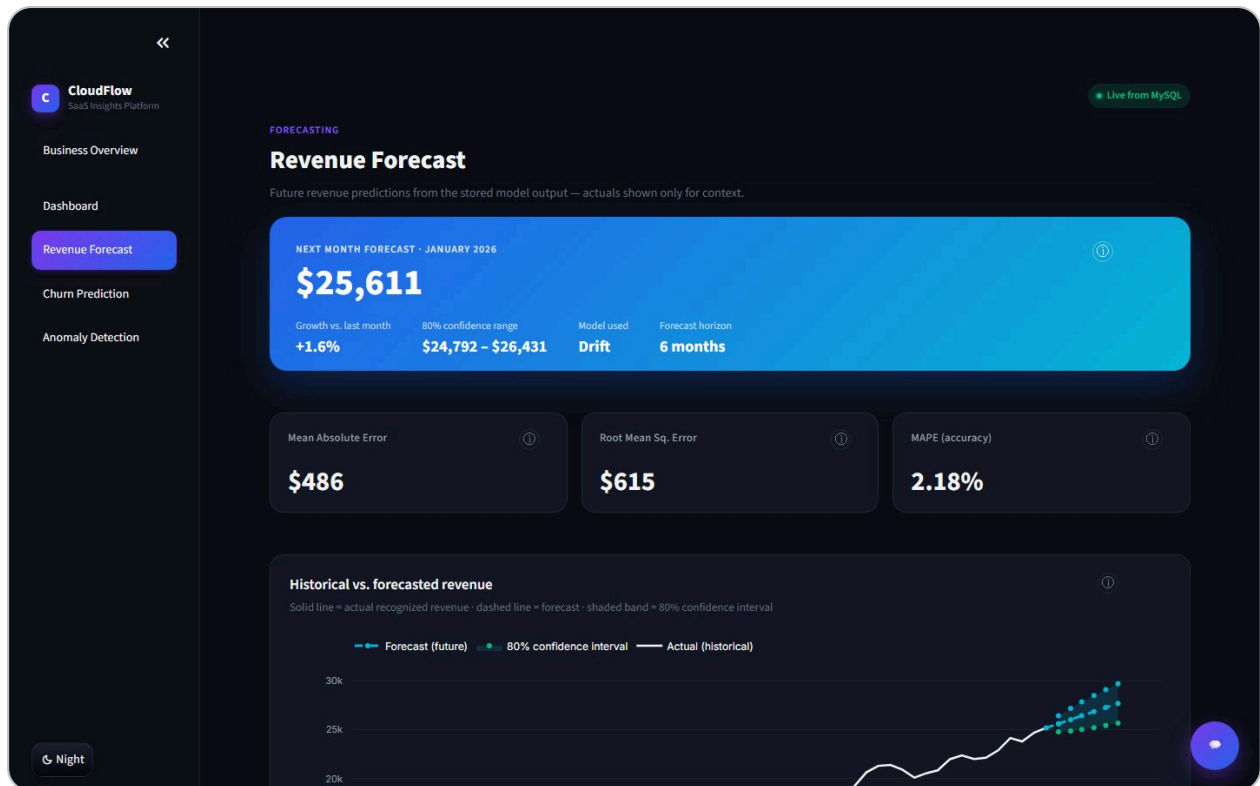
WHAT THE TEAM SEES

One dashboard, actuals and predictions kept strictly apart

The Dashboard page shows only what happened — filterable by date, segment, region, and product, with every KPI backed by a source-and-formula panel on request. The Revenue Forecast page shows only predictions, so no one can mistake a forward-looking number for a historical one.



Dashboard — live KPIs and trends from actual historical data.

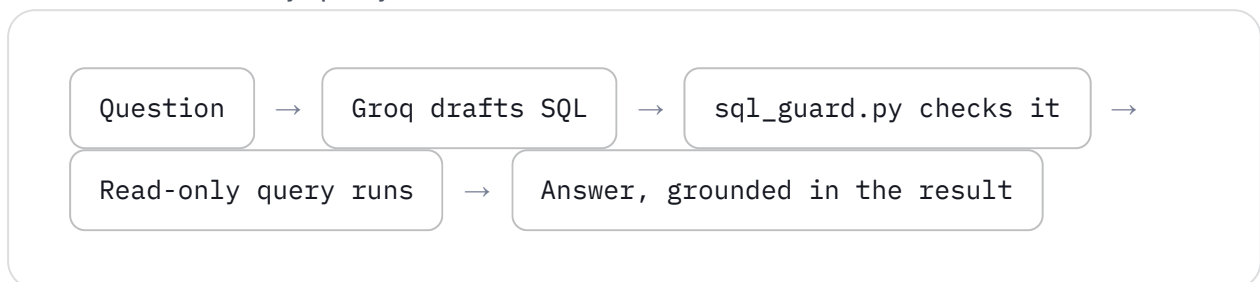


Revenue Forecast — 6-month projection with an 80% confidence band.

ASK IT A QUESTION

A chatbot that isn't allowed to make numbers up

A floating assistant sits on every page. It doesn't generate answers from memory — every response is grounded in a live query against the actual database, and every query is checked before it runs.



`sql_guard.py` rejects anything that isn't a single, read-only `SELECT`, blocks a keyword blacklist, forces the session read-only, and caps results at 200 rows — before the query ever reaches MySQL. If a question falls outside what the data can answer, the assistant says so, rather than guessing.

The stack, end to end

PIPELINE pandas · SQLAlchemy · PyMySQL Ingestion, cleaning, feature engineering	DATABASE MySQL / TiDB Cloud MariaDB locally, TiDB Serverless in production, over TLS
FORECASTING scikit-learn · XGBoost · LightGBM · statsmodels 7 models, walk-forward validated	DASHBOARD Streamlit · Plotly Custom design system, dark/light themes
ASSISTANT Groq API Grounded, SQL-guarded Q&A	INSIGHTS DORMANT Anthropic Claude API Built, not yet wired into the UI
HOSTING Streamlit Community Cloud Deployed from GitHub	VERIFICATION Playwright Live-DOM checks for UI/theme regressions

Built with room to grow

Two nav items already exist in the dashboard for capabilities that aren't built yet — each one opens a plain "coming soon" placeholder rather than a stub that pretends to work.

Churn Prediction	PLANNED
Anomaly Detection	PLANNED

Claude-generated insights on the dashboard

BUILT, NOT WIRED IN

CloudFlow Revenue Analytics — internal project report. Source:

[LejoyAnalytics/datascience_saas_analytics](#).

Generated July 2026.